AyED II

Ingeniería en Sistemas de Información
2° Año -1° Cuatrimestre

Modalidades y Condiciones de Aprobación

Promoción:

- **Aprobar los 2 Parciales con % 80 o más en primera instancia**
- **Asistencia mayor o igual al % 80.**
- **Elaboración y entrega de Prácticos.**

Regularidad:

- **Aprobar los 2 parciales con % 70 o más en primera o segunda instancia (recuperatorio).**
- **Asistencia mayor o igual al % 80.**
- **Elaboración y entrega de Prácticos.**

Parciales

1° Parcial : 27 de Abril

Recuperatorio 1° parcial : 4 de Mayo

2° Parcial : 22 de Junio

Recuperatorio 2° parcial : 26 de Junio


AyED2
AyED2 2026
WhatsApp group

Tipos de Datos

Ventajas de Usar Tipos de Datos:

Prevención y detección de errores.

Ayuda a los desarrolladores a organizar los datos.

Permite identificar propiedades específicas de cada tipo.

Tipos de Datos

Definición:

Un **conjunto de valores**

Un **conjunto de operaciones** que se pueden aplicar sobre esos valores.

..., -2, -1, 0, 1, 2, 3 ...

+, -, *, /, %

Tipos de Datos en Programacion

En los lenguajes de programación se distinguen tres grupos principales:

- **Tipos primitivos**
- **Tipos compuestos**
- **Tipos agregados**

Los tipos primitivos son los **más simples**, mientras que los otros se construyen a partir de ellos.

Estructuras de Datos

Una estructura de datos incluye:

1. Elementos de datos
2. Relaciones entre esos elementos

Array	Registro
Secuencia homogénea	Datos heterogéneos
Relación por posición	Sin relación entre campos

Selección de una Estructura de Datos

Una mala elección puede producir:	**Etapas para seleccionar una estructura de datos:**	**Consideraciones importantes:**
programas más lentos mayor consumo de recursos soluciones poco eficientes	**Analizar el problema** y las restricciones de recursos. **Determinar las operaciones básicas** que se deben soportar (inserción, búsqueda, eliminación). **Seleccionar la estructura de datos** que mejor cumpla los requisitos.	¿Los datos se insertan todos al principio o durante la ejecución? ¿Se pueden eliminar datos? ¿Se necesita acceso secuencial o acceso aleatorio?



Orden

Pasos Claros

No Ambigüedad

¿Que es un ALGORITMO?

Definición:

Un algoritmo es un método, un proceso, un conjunto de instrucciones utilizadas para resolver un problema específico.

Un problema puede ser resuelto mediante muchos algoritmos.

Un algoritmo dado correcto, resuelve un problema definido y determinado.

Propiedades de un Algoritmo

Especificación Precisa de la entrada.

Especificación precisa de cada instrucción.

Exactitud, corrección.

Etapas bien definidas y concretas.

Número finito de pasos.

Un algoritmo debe terminar.

Descripción del resultado o efecto.

Propiedades de un Algoritmo

EJ:

- Escribir una lista de números enteros positivos.
- Ordenar las letras del Abecedario.

Eficiencia Algorítmica

EFICIENCIA ≠ EFICACIA

VIAJAR DE VM a Bs As

- Caminando
- Auto
- Avión

ALGORITMOS

UN PROBLEMA ≠ UNA SOLUCIÓN

EJ: Buscar un número en una lista

Método A: uno por uno

Método B: dividir por mitades

Complejidad Algorítmica

Entonces, ¿qué es eficiencia?

La eficiencia de un algoritmo es la propiedad mediante la cual un algoritmo debe alcanzar la solución al problema en el tiempo más corto posible o utilizando la cantidad más pequeña posible de recursos físicos y que sea compatible con su exactitud o corrección.

Complejidad Algorítmica

Cómo medimos qué Algoritmo es Mejor?



Para Recordar

1. Un problema es una función o asociación de entradas con salidas.

2. Un algoritmo es una receta para resolver un problema cuyas etapas son concretas y no ambiguas.

3. El algoritmo debe ser correcto y finito, y debe terminar para todas las entradas.

4. Un programa es una “ejecución” (instanciación) de un algoritmo en un lenguaje de programación de computadora.

Complejidad Algorítmica

Análisis de algoritmos: permite medir la dificultad inherente de un problema. Esta característica nos permitirá comparar algoritmos para la resolución de problemas en términos de eficiencia.

La eficiencia como factor **espacio-tiempo** debe estar estrechamente relacionada con la buena calidad, el funcionamiento y la facilidad de mantenimiento del programa.

Eficiencia Algorítmica

Estrategia Empírica: consiste en programar los algoritmos y ejecutarlos en un procesador sobre algunos ejemplares de prueba.

Estrategia Matemática: consiste en determinar matemáticamente la cantidad de recursos (tiempo, espacio, etc.) que necesitará el algoritmo en función del tamaño del ejemplar considerado.

Eficiencia Algorítmica

Análisis A Priori: entrega una función que limita el tiempo de cálculo de un algoritmo. Consiste en obtener una expresión que indique el comportamiento del algoritmo en función de los parámetros.

Prueba A Posteriori (experimental o empírica): se recogen estadísticas de tiempo y espacio consumidas por el algoritmo mientras se ejecuta.

Eficiencia Algorítmica

	Análisis A Priori (teórico)	**Prueba A Posteriori (empírico)**
	Estudia el algoritmo **antes de implementarlo** mediante fórmulas	Evalúa el algoritmo **ya implementado**, midiendo ejecución real
Pros	- No depende de máquina ni lenguaje - Se puede hacer sin programar - Evita implementar algoritmos ineficientes - Sirve para cualquier tamaño de entrada - Útil en etapa de diseño	- Resultados reales y concretos - Tiene en cuenta el entorno (hardware, compilador, etc.) - Permite validar el comportamiento práctico
Contras	- Es una aproximación (no siempre refleja la realidad exacta) - No considera detalles del sistema (cache, CPU, etc.) - Puede ser complejo de calcular	- Depende de máquina, lenguaje y compilador - Requiere implementación previa - Consume tiempo y recursos - Resultados no generalizables
Cuándo usar	Diseño y comparación de algoritmos	Validación y medición real

Tamaño de Datos

Que es?: Es la **variable (n)** que usamos para medir la complejidad de un algoritmo.

Tipo de problema	Tamaño (n)
Arreglo	Número de elementos
Cadena de texto	Cantidad de caracteres
Número	Cantidad de dígitos
Matriz	Filas, columnas o total de elementos
Ordenamiento	Número de elementos a ordenar
Recursividad	Cantidad de llamadas recursivas
Grafos	Número de nodos y aristas

Análisis Peor Caso, Mejor Caso y Caso Promedio

Tipo	Qué mide?
Peor caso	El **mayor tiempo posible** (caso más desfavorable)
Mejor caso	El **menor tiempo posible** (caso ideal)
Caso promedio	El **tiempo esperado** con entradas típicas

Eficiencia Algorítmica

f (n) = eficiencia

Donde n = cantidad de datos de entrada. (tamaño del problema)

La eficiencia del algoritmo se examina como una función del número de elementos que tienen que ser procesados.

Eficiencia Algorítmica

```
1    i = 1
2    iterar (i <= n)
            1    código de la aplicación
            2     i = i + 1
3    fin _ iterar
```

f (n) = n

```
1    i = 1
2    iterar (i <= n)
            1    código de la aplicación
            2     i = i + 2
3    fin _ iterar
```

f (n) = n/2

Eficiencia Algorítmica

```
1    i = 1;
2    while (i < n){
            1    //código de la aplicación
            2     i = i * 2;
3    }
```

f (n) = log(n)

```
1    i = n;
2    while (i >1){
            1    //código de la aplicación
            2     i = i / 2;
3    }
```

f (n) = log(n)

Eficiencia Algorítmica

```
1    i = 0;
2    while (i < n){
             int j = 1;
             while ( j < n){
                    j = j*2;
             }
             i++;
3    {
```

f (n) = n log(n)

Análisis de Rendimiento

Complejidad Espacial:

Cantidad de memoria que se necesita para ejecutarlo hasta la terminación.

Complejidad Temporal:

Cantidad de tiempo de computadora que se necesita para ejecutarlo.

Análisis de Rendimiento

```
double serie(double x, int n) {
    double s;
    int i;
    s = 0.0;                                        // tiempo t1

                for (i = 1; i <= n; i++){           // tiempo t2

                        s += i*x;                   // tiempo t3
                }

     return s;                                      // tiempo t4
 }
```

Análisis de Rendimiento

La función de rendimiento **T(n)** *(complejidad temporal)*, para representar el número de unidades de tiempo tomadas por un programa o algoritmo para cualquier entrada de tamaño n.

La función **T(n)** del método es:

$$T(n) = t1 + n*t2 + n*t3 + t4$$

$$Tmin(n) <= T(n) <= Tmax(n)$$

Análisis de Rendimiento

Las operaciones básicas son las que realiza el procesador en tiempo acotado por una constante, por ejemplo:

1. Operaciones aritméticas básicas
2. Asignaciones de tipo predefinidos
3. Saltos (llamadas a funciones, procedimientos y retorno)
4. Comparaciones lógicas
5. Acceso a estructuras indexadas básicas (vectores y matrices)

Métodos para Medir/Analizar Rendimiento

Benchmarking : La técnica de benchmark considera una colección de entradas típicas representativas de una carga de trabajo para un programa.

Profiling : Consiste en asociar a cada instrucción de un programa un número que representa la fracción del tiempo total tomada para ejecutar esa instrucción particular.

Análisis : Consiste en agrupar las entradas de acuerdo a su tamaño, y estimar el tiempo de ejecución del programa en entradas de ese tamaño, T(n).

Concepto de Complejidad

La complejidad (o costo) de un algoritmo es una medida de la cantidad de recursos (tiempo, memoria) que el algoritmo necesita. La complejidad de un algoritmo se expresa en función del tamaño (o talla) del problema.

La función de complejidad tiene como variable independiente el tamaño del problema y sirve para medir la complejidad (espacial o temporal). Mide el tiempo/espacio relativo en función del tamaño del problema.

Función de Complejidad

El comportamiento de la función determina la eficiencia. No es única para un algoritmo: depende de los datos. Para un mismo tamaño del problema, las distintas presentaciones iniciales de los datos dan lugar a distintas funciones de complejidad. Es el caso de una ordenación si los datos están todos inicialmente desordenados, parcialmente ordenados o en orden inverso.

Función de Complejidad

La complejidad se expresa como una función:

$$T(n)$$

donde:

- **n** = tamaño del problema
- **T(n)** = costo del algoritmo *(tiempo)*

Ejemplo

Supongamos que tenemos una máquina que puede procesar 1000 operaciones por hora, y compramos una 10 veces mas rapida:

Complejidad	n inicial	n nuevo	Mejora
$O(n)$	10.000	100.000	×10
$O(n \log n)$	1.000	8.000	×8
$O(n^2)$	100	316	×3.16
$O(2^n)$	13	16	+3

Notación Big-O

La notación O indica el límite superior del tiempo de ejecución de un algoritmo, en lugar de expresar un tiempo exacto como 4n + 10, se simplifica a O(n), representando su crecimiento e ignorando constantes.

Esto permite abstraerse de factores como el lenguaje, la máquina o el programador, enfocándose en el comportamiento del algoritmo para tamaños de entrada grandes (análisis asintótico).

Notación Big-O

Sea T(n) el tiempo de ejecución de un algoritmo en función del tamaño de entrada n. Se dice que T(n) es O(g(n)) cuando la función g(n) actúa como un límite superior de su crecimiento.

Formalmente, esto significa que existen una constante $c > 0$ y un valor n_0 tal que, para todo $n \geq n_0$, se cumple:

$$\mathbf{T(n) \leq c \cdot g(n)}$$

Determinar Notación O

O(f(n)) indica el nivel de complejidad de un algoritmo. Para representarlo, se usa la forma más simple de una función que crezca de la misma manera.

Donde:

f(n) *cantidad de operaciones que realiza un algoritmo*

Ejemplo:

$$f(n) = 3n^2 + 5n + 10$$

Determinar Notación O

1. En cada término, establecer el coeficiente del término en 1.
2. Mantener el término mayor de la función y descartar los restantes.

$f(n) = 3n^2 + 5n + 10$ entonces $O(f(n)) = n^2$

Orden de complejidad

Complejidad	Nombre	Descripción
O(1)	Constante	No depende del tamaño de entrada
O(log n)	Logarítmica	Crece muy lento (divide el problema)
O(n)	Lineal	Crece proporcional a n
$O(n^2)$	Cuadrática	Dos bucles anidados típicamente
$O(n^a)$	Polinomial	Crece más rápido que n^2 ($a > 2$)
$O(a^n)$	Exponencial	Crece muy rápido (combinaciones)
O(n!)	Factorial	Crecimiento extremo (todas las permutaciones)

Orden de Complejidad

Complejidad	Ejemplo
O(1)	Acceder a `arr[i]`
O(log n)	Búsqueda binaria
O(n)	Recorrer un arreglo
O(n log n)	Merge Sort, Quick Sort (promedio)
$O(n^2)$	Bubble Sort, Selection Sort
$O(n^a)$	3 bucles anidados (ej: n^3)
$O(a^n)$	Fuerza bruta de contraseñas
O(n!)	Permutaciones, problema del viajante

Notación Omega (Ω)

Ω(g(n)) indica un límite inferior, el algoritmo crece al menos como g(n)

$$T(n) \geq c \cdot g(n), \text{ para } n \geq n_0$$

Notación Theta (Θ)

Θ(g(n)) indica un límite ajustado, El algoritmo crece exactamente como g(n)

$$c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n), \text{ para } n \geq n_0$$

Costo Unitario

Se asume que las operaciones básicas tienen costo constante

- asignaciones
- sumas
- comparaciones

todas valen:

O(1)

Regla de suma

Si un algoritmo tiene:

- $A_1 \rightarrow O(f(n))$
- $A_2 \rightarrow O(g(n))$

$T(n) = O(max(f(n), g(n)))$

for (...) → $O(n)$
for (...) → $O(n^2)$

$O(n^2)$

Regla de producto (anidación)

Si tenemos::

- un bloque que se repite dentro de otro

$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$

```
for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
}}
```

$O(n^2)$

Regla de condición (if)

Estructura condicional:

```
if (...) {
   A
 } else {
    B
}
O(max(A, B))
```

Regla de iteración (for / while)

Bucles:

```
for (i=0; i<n; i++) {
   A
}
```

$n \times costo(A)$

Ejemplo

```
for (int i = 0; i < n; i++) {          // (1)

    if (i % 2 == 0) {                  // (2)

        for (int j = 0; j < n; j++) {  // (3)
            x++;                       // (4)
        }

    } else {
        y++;                           // (5)
    }
}
```

Paso a Paso

(4) Operación básica O(1)

(3) Bucle interno n × O(1) = **O(n)**

(5) Rama else O(1)

(2) Condicional O(max(n, 1)) = **O(n)**

(1)Bucle externo n × O(n) = **O(n²)**

O(n²)